

POLITECNICO
MILANO 1863

Software Engineering 2

Introduction to Software Design

Software Architecture as a set of structures and role of UML

Implicit vs explicit architectures

Exercise: reasoning on architectural documentation

M Camilli, E Di Nitto, M Rossi

What Is Software Design?

POLITECNICO MILANO 1863

- Software Design = Deciding **how** a system fulfils its requirements
- Considers:
  - Functional & non-functional requirements
  - Technological, budget, and resource constraints
  - Organizational context
- It is decision-making under constraints, thus, it is not just about structure, but trade-offs.

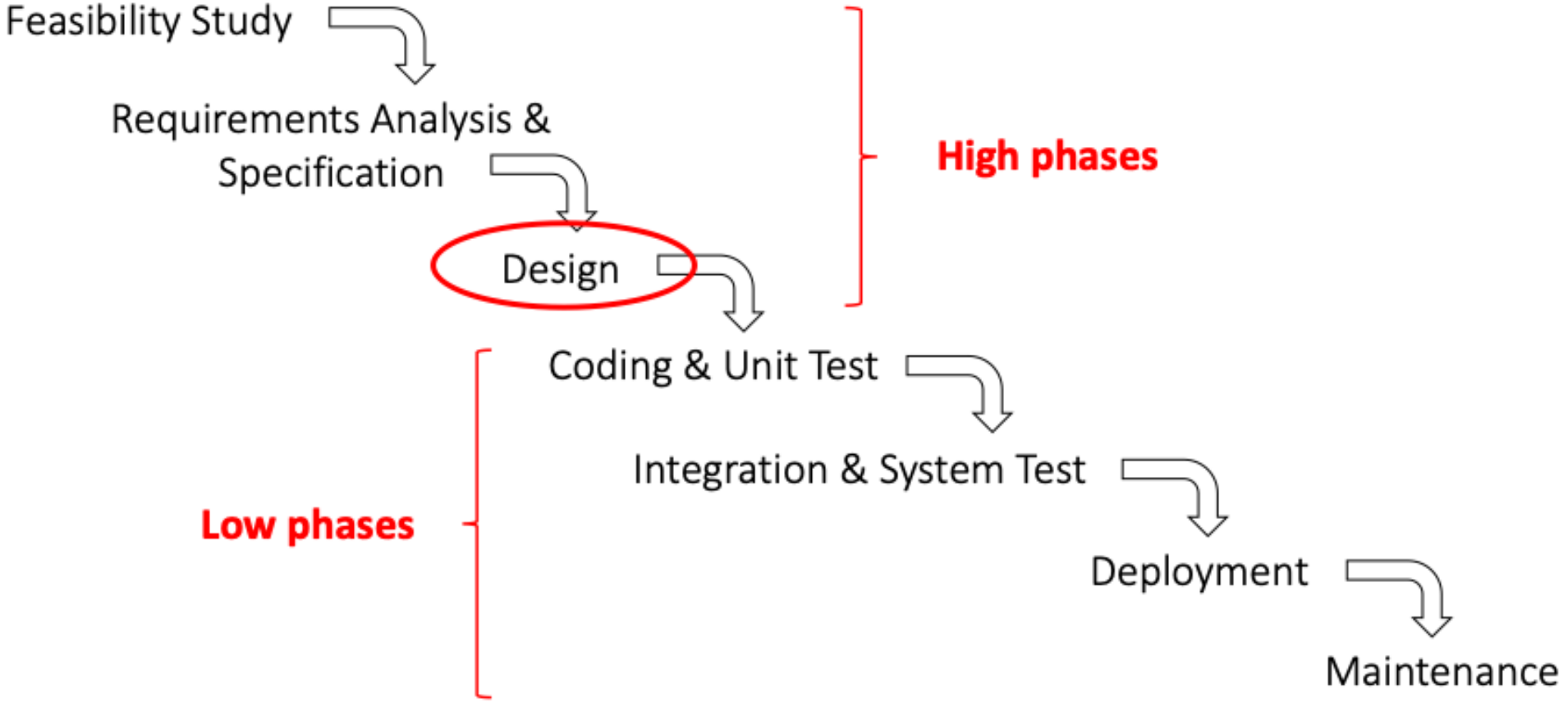
Why design decisions matter deeply? Case study: the Ariane 5 failure



On June 4, 1996, the Ariane 5 launcher exploded 37s after liftoff

- Trigger: Incorrect 64-bit float to 16-bit integer conversion
- Root causes: **design flaws** in the Inertial Reference System (IRS) and its interaction with the On-Board Computer (OBC):
  - **Inappropriate Reuse**: IRS was designed for Ariane 4
  - **Inappropriate Exception Handling:** the two replicas of the IRS both faced the incorrect conversion and shutdown leaving the OBC with no information
  - **Vague Interaction Protocol:** The OBC wrongly interpreted a **diagnostic bit pattern** sent by the failed IRS as flight data, leading it to command booster deflection

Design in the software lifecycle



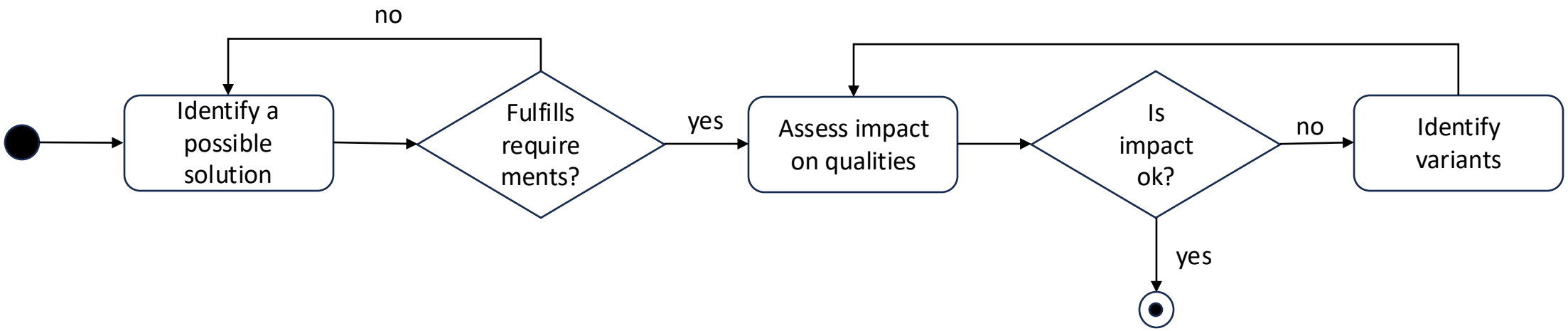
What is a Software Architecture?

POLITECNICO MILANO 1863

- The Software Architecture (SA) of a system is the **set of structures** needed to reason about the system. These structures comprise software **elements**, **relations** among them, and **properties** of both. [Bass et al. 2021]
- SA as a tool to reason about systems
- SA composed of a set of structures

[Bass et al. 2021] *Bass, Len, et al. Software Architecture in Practice, Pearson Education, Limited, 2021.*

Architectural reasoning workflow

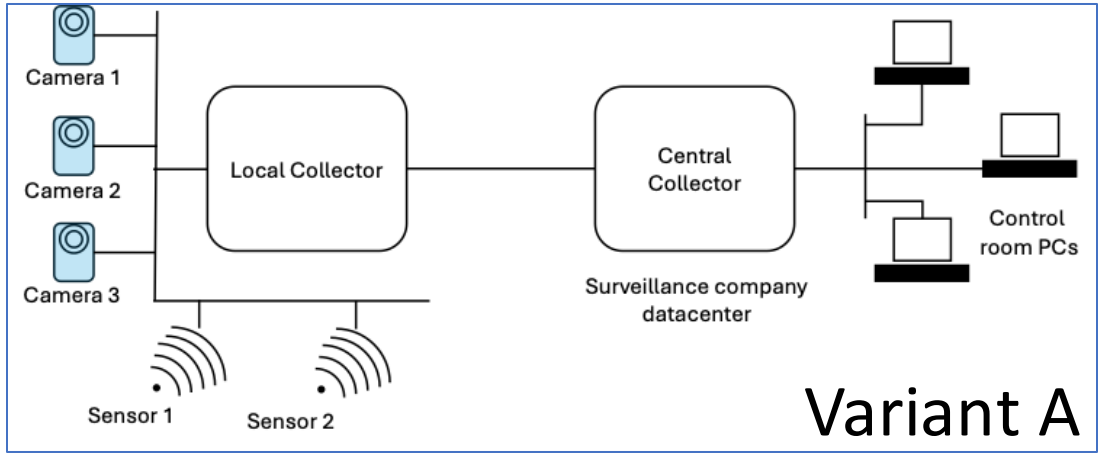


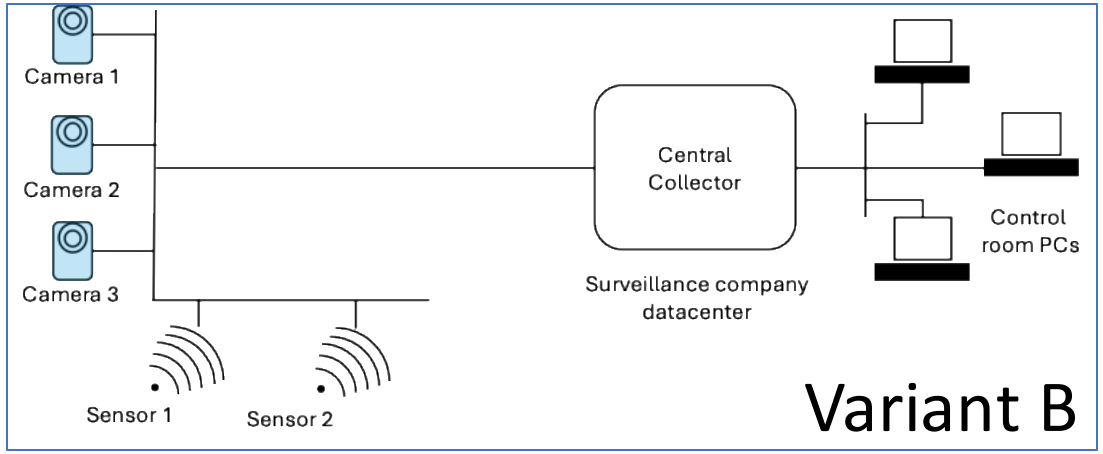
Example: Surveillance System

POLITECNICO MILANO 1863

- We want to develop a software system that allows a surveillance company to visualize in its control room the surroundings of some public building and to receive alerts when someone enters the building in period when it is closed. Also assume that cameras and movement sensors have already been installed and can be controlled remotely via software.

Example: Surveillance System





- **Trade-offs**
  - Latency vs. Flexibility
  - Future extensibility vs. simplicity

Architecture as Multiple Structures

POLITECNICO MILANO 1863

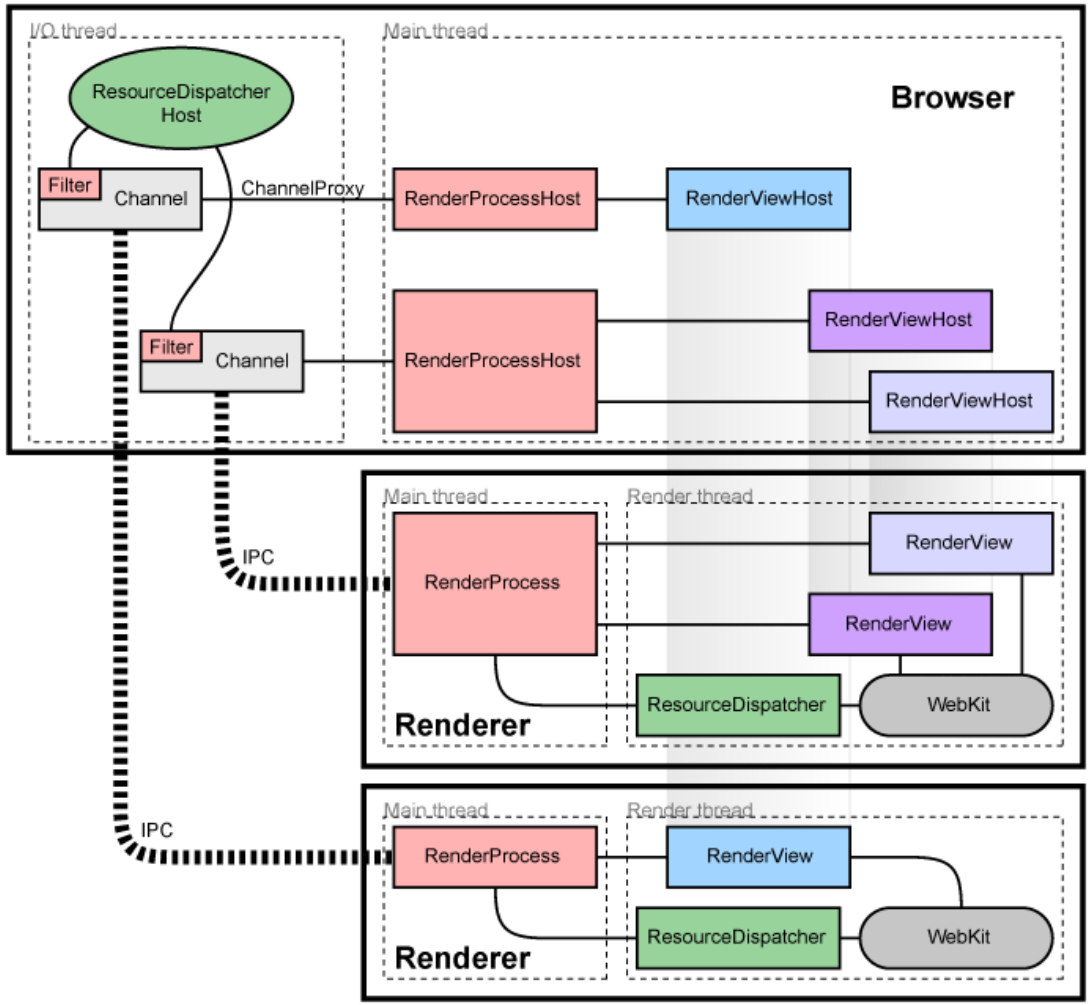
- In complex systems multiple structures coexist – Example: design of a building includes
  - The organization of structural elements (e.g., columns, beams)
  - The electrical system
  - The heating structure
  - The water system
- Software architecture = **Set of structures/views**
  - Component & Connector (runtime organization)
  - Module (implementation structure)
  - Deployment (execution environment)

Component-and-connector (C&C) structures

- Describe how the system is structured as a set of elements that have runtime behavior (components) and interactions (connectors).
- **Components**
  - Principal units of computation
  - Examples: clients, servers, services, …
- **Connectors**
  - Communication means
  - Examples: request-response mechanisms, pipes, asynchronous messages, shared memory …

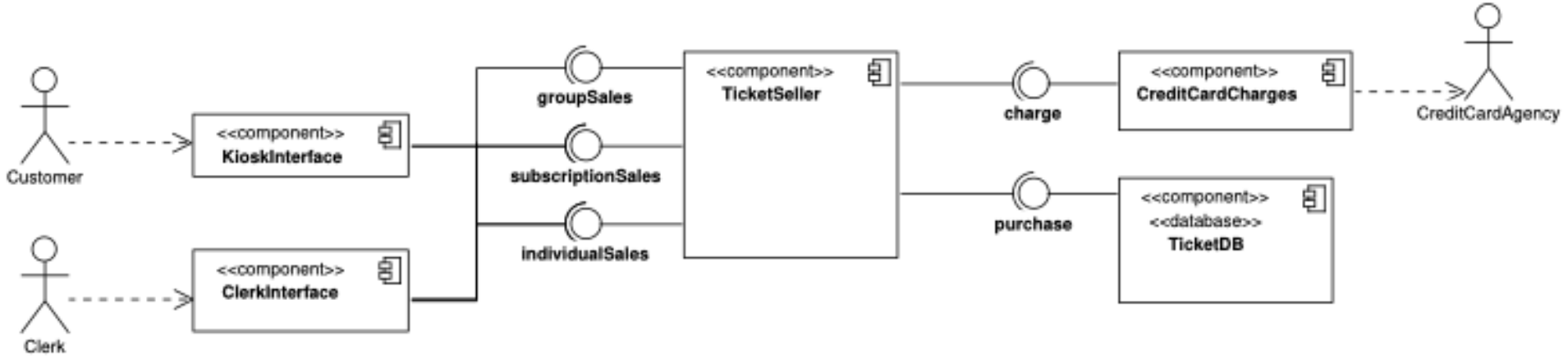
Description of C&C structures – Informal notation

- From the Chromium documentation https://www.chromium.org/developers/design-documents/multi-process-architecture

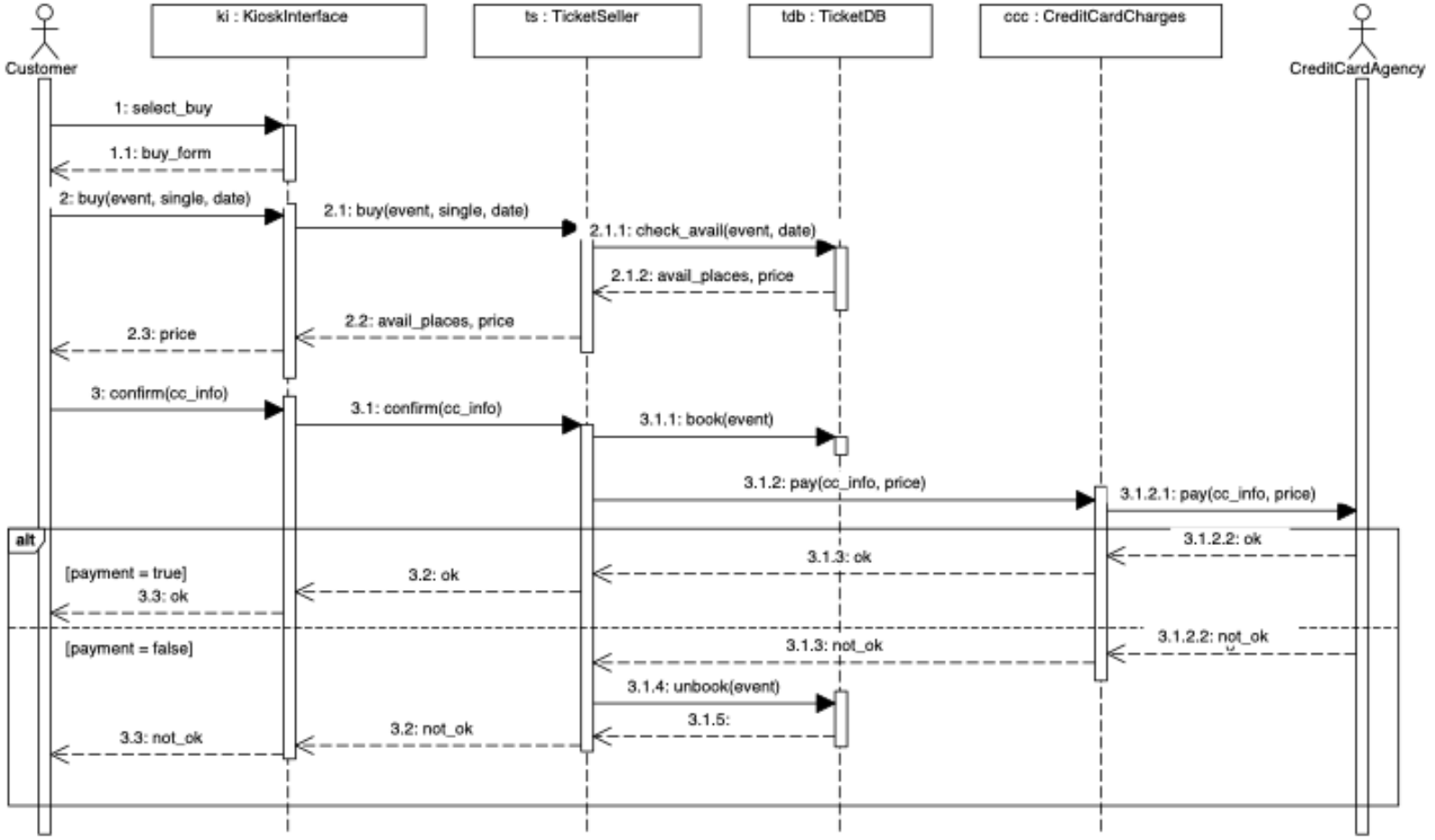


Description of C&C structures – UML component diagram

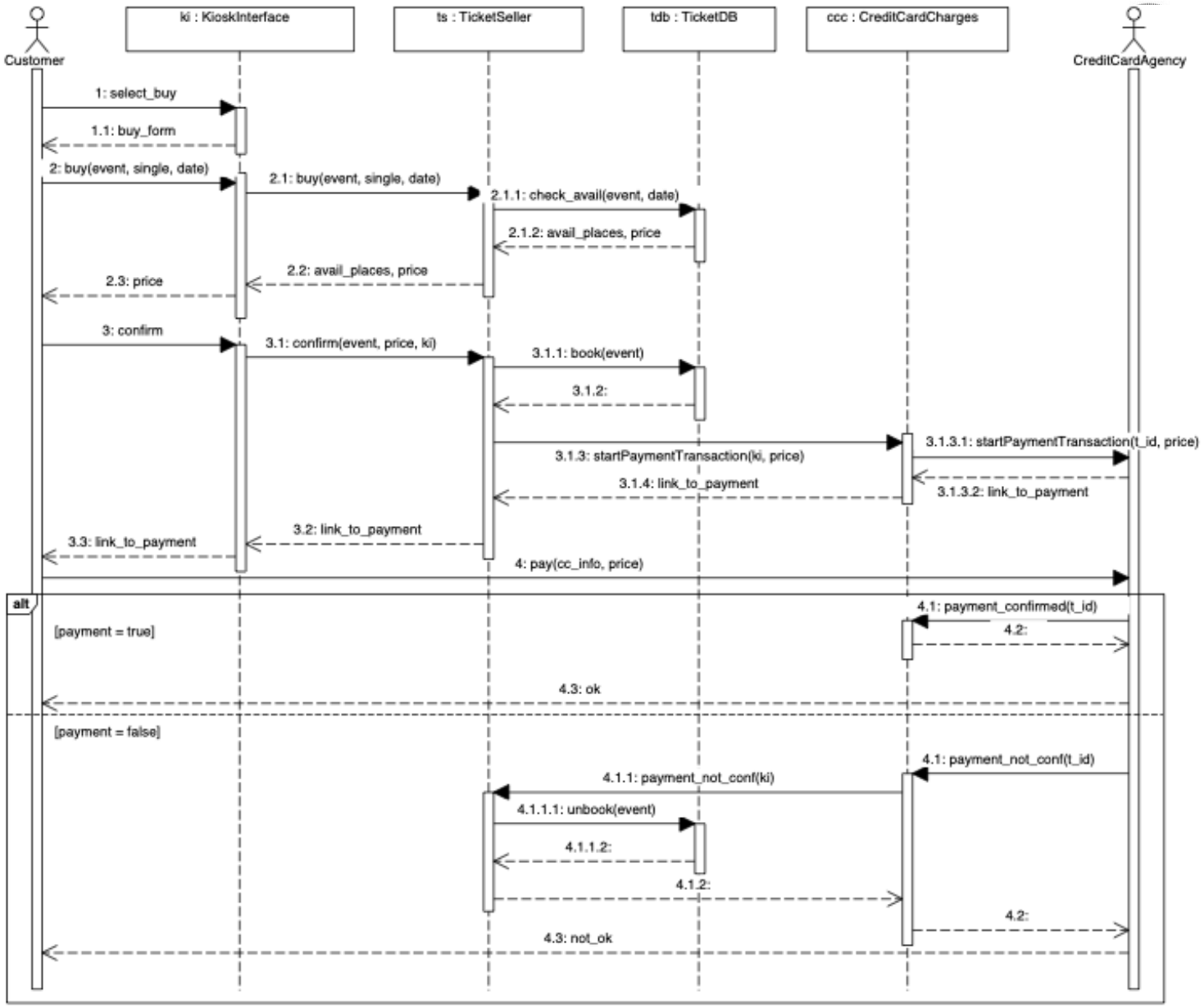
- Example of a ticket selling system usable by customers through public kiosks and by clerks through their computers



Description of C&C structures – UML sequence diagram (variant 1)



Description of C&C structures – UML sequence diagram (variant 2)



Purpose of C&C structures

POLITECNICO MILANO 1863

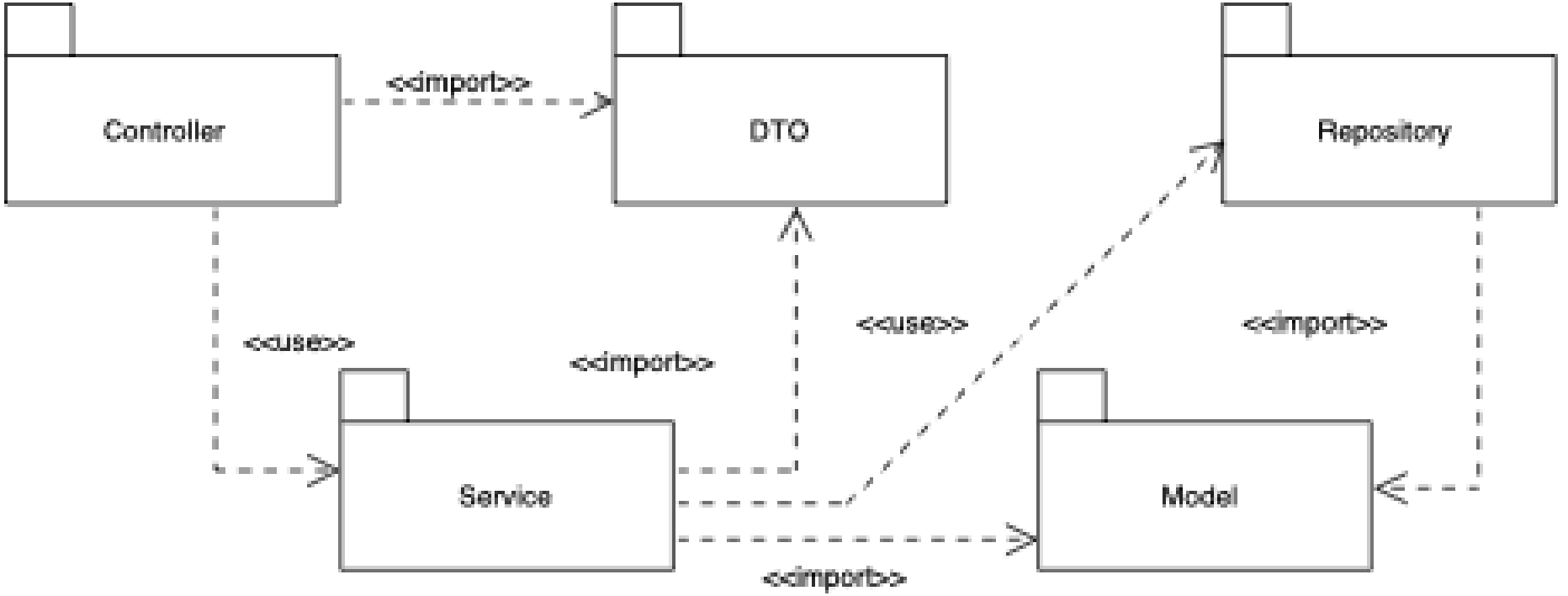
- Essential for understanding how the software will behave at runtime
- It helps make decisions on component-specific responsibilities and data flow
  - E.g., reflect on the pros/cons of the two variants in the previous slides
- Also, allow us to study runtime properties such as availability and performance
  - E.g., TicketSeller is a bottleneck in our system. Could we replicate it? What are the implications of replication?

Module structures

- Show how a system is structured as a set of code or data units that have to be procured or constructed, together with their relations
- Examples of modules: packages, classes, functions, libraries, layers, database tables…
- Modules constitute implementation units that can be used as the basis for work splitting (identifying functional areas of responsibility)
- Typical relations among modules:
  - uses, is-a (generalization), is-part-of

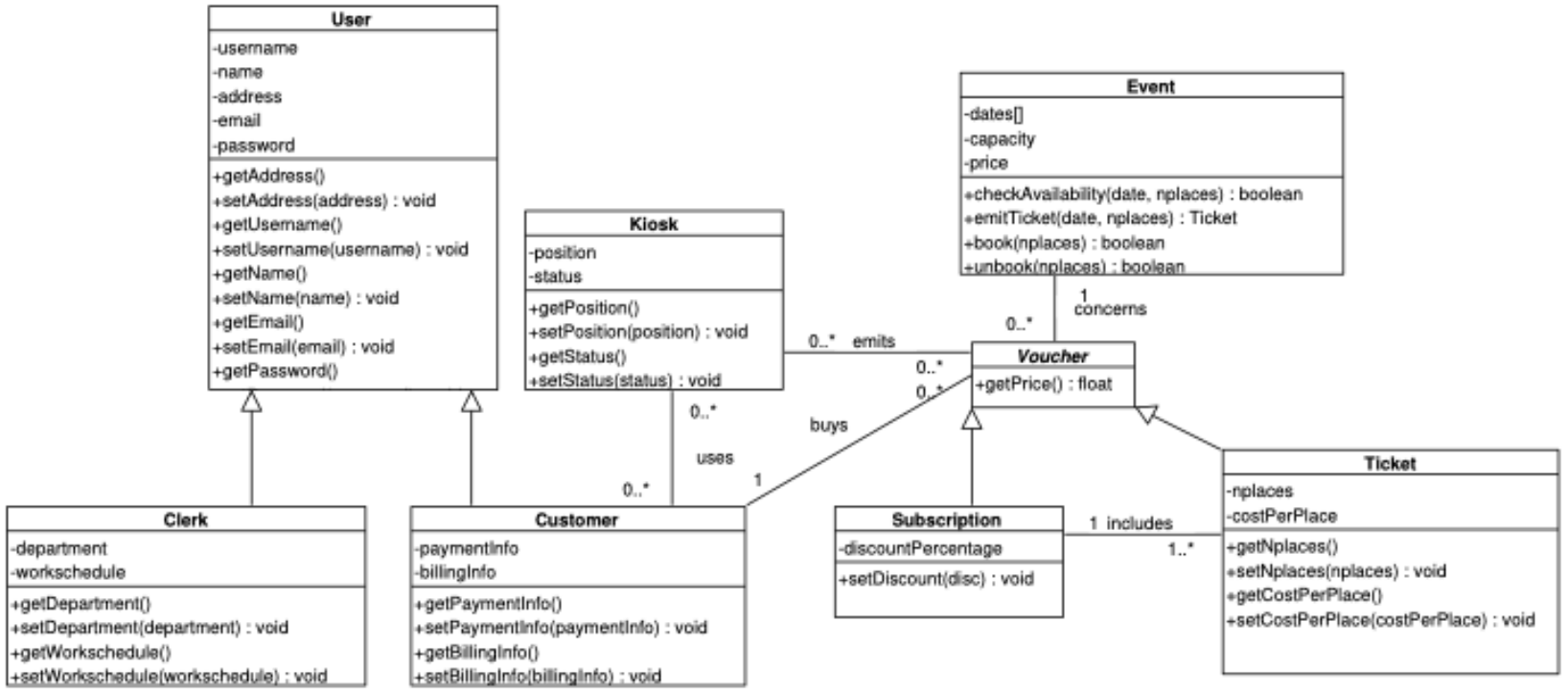
Description of module structures

- Example of package diagram describing the internals of TicketSelling



Description of module structures

- Classes in the TicketSelling Model package (class diagram)



Purpose of module structures

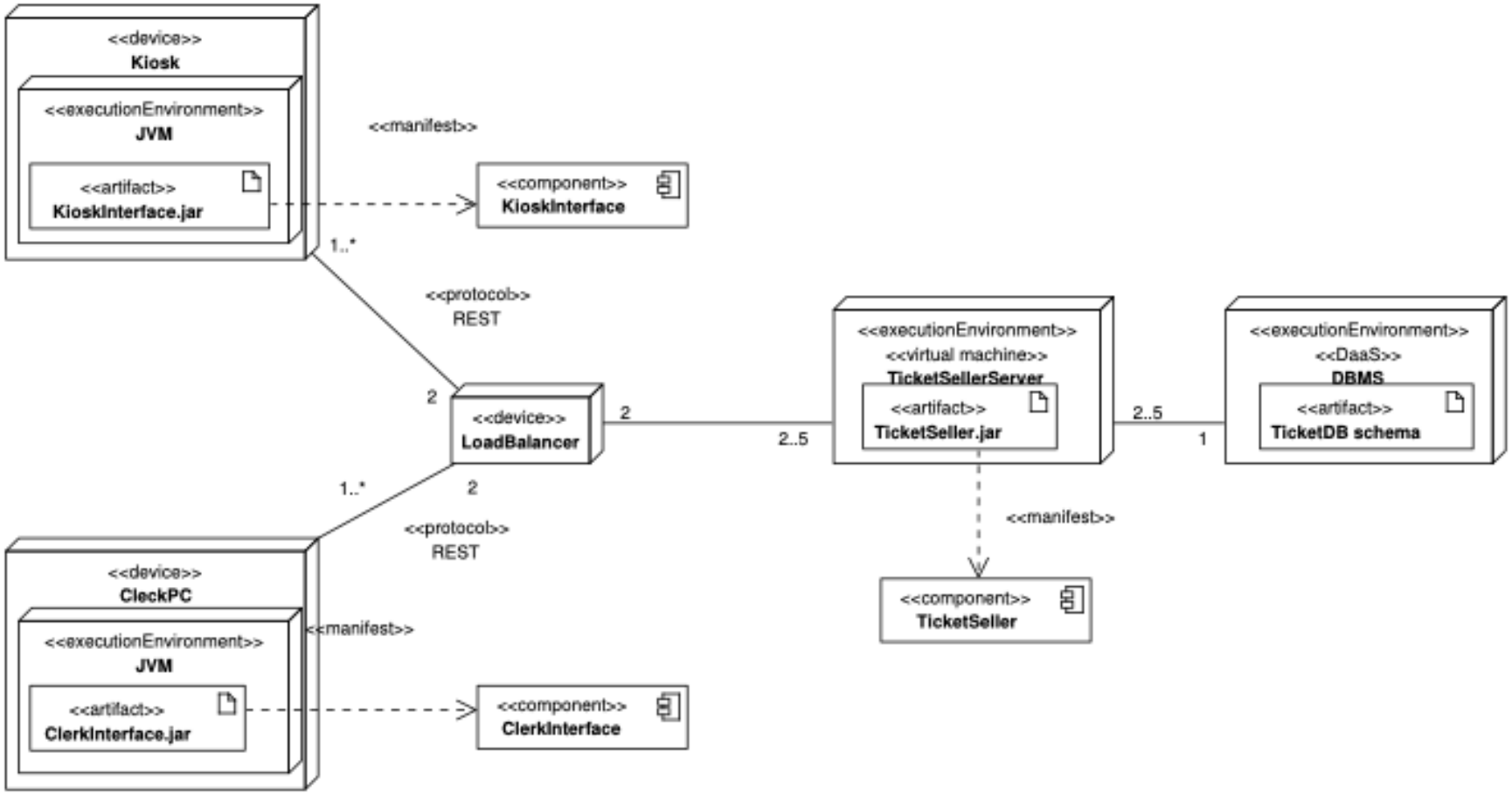
POLITECNICO MILANO 1863

- Supports planning the **implementation phase** and distributing work among team members
- Crucial for reasoning on **maintainability**: dependencies help assess the impact of required changes

Deployment structure

- Identifies the **topology of the infrastructure** needed for execution and how architectural elements are mapped onto it
- Main elements:
  - Devices
  - Execution environments (VMs, interpreters, containers)
  - Supporting hardware (load balancers, firewalls)

Description of deployment structure – UML deployment diagram



Purpose of deployment structure

POLITECNICO MILANO 1863

- Provides information to
  - assess non-functional requirements such as **performance efficiency, reliability, and security**
  - identify and address potential **bottlenecks** (e.g., justifying the use of a LoadBalancer to manage replication)
  - reasoning about security, helping to determine where to install firewalls

Coherence between structures

POLITECNICO MILANO 1863

- Components implemented by modules
- Modules become artifacts that are deployed in execution environments
- At runtime, artifacts become component instances

Implicit vs explicit architectures

POLITECNICO MILANO 1863

- **Implicit:** Design exists only in code and in the mind of the main developers
- **Explicit:** Documented, shareable architecture

Implicit vs explicit architectures: example

POLITECNICO MILANO 1863

```
import java.io.*; import java.net.*; import java.time.LocalDateTime;
public class TimeServer {
public static void main(String[] args) {
  int port = Integer.parseInt(args[0]);
  try (ServerSocket serverSocket = new ServerSocket(port)) {
     System.out.println("TimeServer running on port " + port);
 while (true) {
   try (
      Socket clientSocket = serverSocket.accept();
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
   ){
     String currentTime = LocalDateTime.now().toString();
     out.println(currentTime);
     System.out.println("Sent time to client: " + currentTime);
    }}
 } catch (IOException e) {
  e.printStackTrace();
 }  }}
```

Implicit vs explicit architectures: example

POLITECNICO MILANO 1863

```
import java.io.*; import java.net.*;
public class TimeClient {
public static void main(String[] args) {
  String serverAddress = args[0]
  int port = Integer.parseInt(args[1]);
  try (
    Socket socket = new Socket(serverAddress, port);
    BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
  ) {
     String serverTime = in.readLine();
     System.out.println(""Current time from server: " + serverTime);
    }}
  } catch (IOException e) {
   e.printStackTrace();
}   }}
```

Implicit vs explicit architectures

POLITECNICO MILANO 1863

- Reverse engineering can recover structure
- But it becomes quickly **cumbersome and time-consuming** with the growth of the codebase

Ingredients of the Architecting Process

POLITECNICO MILANO 1863

- Functional requirements/Use Cases help identify distinct functional areas to map into components
  - e.g., UC: “search for product” naturally leads to the ProductCatalogService component
- Non-functional requirements guide architectural decisions
  - e.g., a requirement for high availability for searching requires considering replication of the ProductCatalogService

Wrap-up

POLITECNICO MILANO 1863

- Key Takeaways
  - Design = reasoning under constraints
  - Architecture = multiple consistent structures
  - Explicit design improves quality and collaboration

POLITECNICO
MILANO 1863

Software Design

Exercise: Reasoning on architectural documentation — WE2 exam of Feb 10, 2023

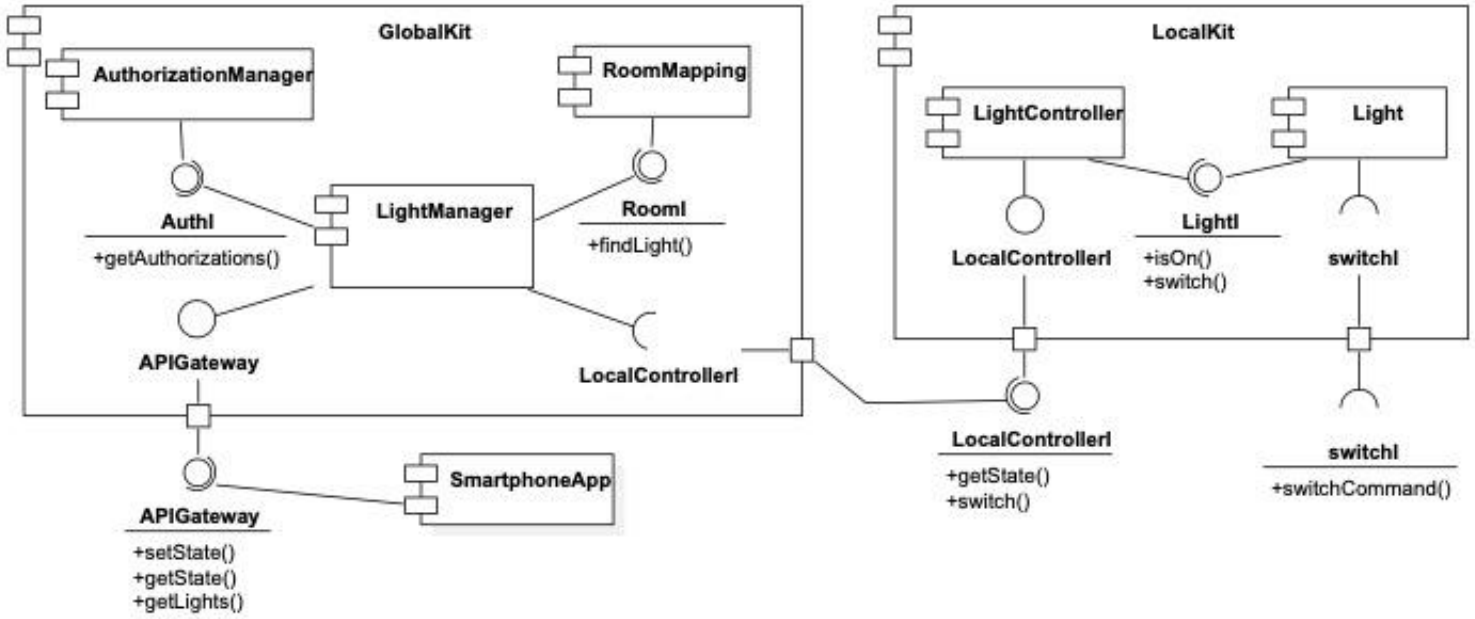
SmartLightingKit

POLITECNICO MILANO 1863

- SmartLightingKit is a system expected to manage the lights of a potentially big building composed of many rooms (e.g., an office space).
- Each room of the building may have one or more lights.
- The system shall allow the lights to be controlled – either locally or remotely – by authorized users. Local control is achieved through terminals installed in the rooms (one terminal per room). Remote control is realized through a smartphone application, or through a central terminal installed in the control room of the building.
- While controlling the lights of a room, the user can execute one or more of the following actions: turn a light on/off at the current time, at a specified time, or when certain events happen (e.g., a person enters the building or a specific room). Moreover, users can create routines, that is, scripts containing a set of actions.
- SmartLightingKit manages remote control by adopting a fine-grained authorization mechanism. This means there are multiple levels of permissions that may even change dynamically.
- System administrators can control the lights of every room, install new lights, and remove existing ones. Administrators can also change the authorizations assigned to regular users. These last ones can control the lights of specific rooms. When an administrator installs a new light in a room, he/she triggers a pairing process between the light and the terminal located in that room. After pairing, the light can be managed by the system.

SmartLightingKit – part 1

- This diagram describes the portion of the SmartLightingKit system supporting lights turning on/off and lights status checking.



- What can we infer from the analysis of this diagram?

SmartLightingKit – part 2

POLITECNICO MILANO 1863

- The diagram is complemented by the following description
  - `GlobalKit` is the component installed in the central terminal. It can be contacted through the `APIGateway` interface by the `SmartphoneApp` component representing the application used for remote control. `GlobalKit` includes:
    - `RoomMapping`, which keeps track, in a persistent way, of lights' locations within the building's rooms;
    - `AuthorizationManager`, which keeps track, in a persistent way, of the authorizations associated with each user (for simplicity, we assume that users exploiting the operations offered by the `APIGateway` are already authenticated through an external system and include in their operation calls a proper token that identifies them univocally); and
    - `LightManager`, which coordinates the interaction with all `LocalKit` components.
  - Each `LocalKit` component runs on top of a local terminal. Also, each `LocalKit` exposes the `LocalControllerI` interface that is implemented by its internal component `LightController`, which controls the lights in the room. Each light is represented in the system by a `Light` component which interacts with the external system operating the light through the `switchCommand` operation offered by the latter.

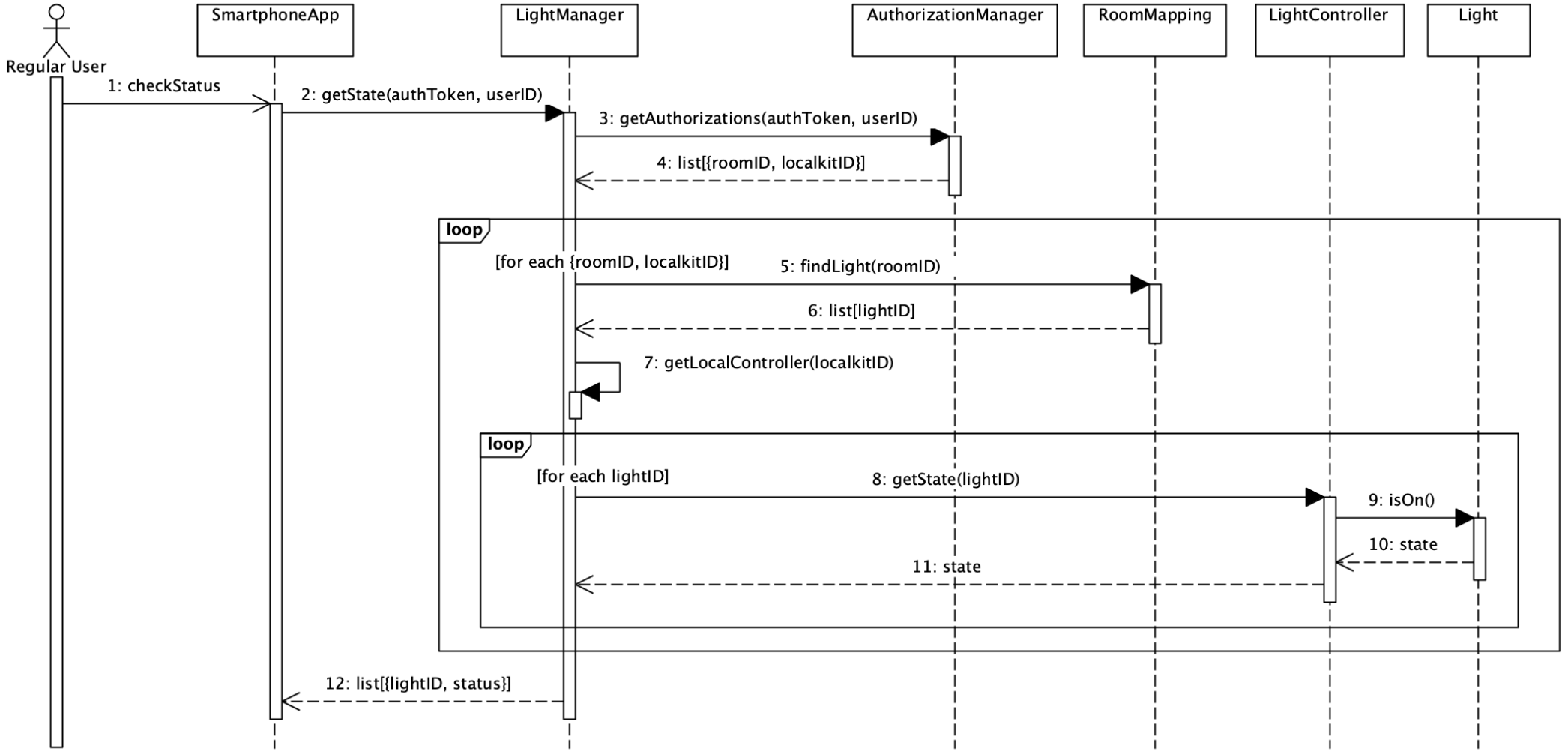
SmartLightingKit – part 2 (cont.)

- Q1:
  Analyze the operations offered by the components shown in the diagram and identify proper input and output parameters for each of them.
- Q2:
  Write a UML Sequence Diagram illustrating the interaction between the software components when a regular user wants to use the smartphone application to check whether he/she left some lights on (among the lights he/she can control).

Operations

- **APIGateway**
- *getLights*: input = userID; output = list[lightID]
- *getState*: input = userID; output = list[(lightID, state)]
- *setState*: input = userID, lightID, state; output = none
- **AuthI**
- *getAuthorizations*: input = userID; output = list[(roomID, localKitID)]
- **RoomI**
- *findLight*: input = roomID; output = list[lightID]
- **LocalControllerI**
- *switch*: input = lightID; output = none
- *getState*: input = lightID; output = state
- **LightI**
- *isOn*: input = none, output = True/False
- *switch*: input = none, output = none
- **SwitchI**
- *switchCommand*: input = none; output = none
- **Note**
- State = On/Off;
- All the operations of `APIGateway`, `AuthI`, `RoomI`, `LocalKitI` receive as input also the authentication token.

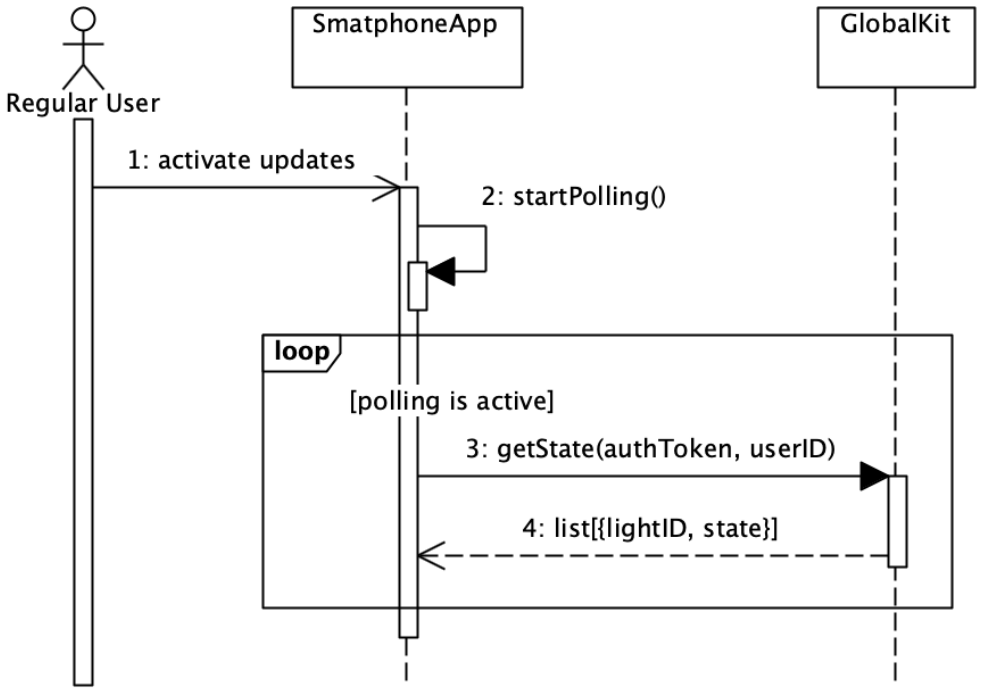
Sequence diagram



SmartLightingKit – part 3

- Assume that, at a certain point, the following new requirement is defined:
  - **NewReq**: *"The smartphone application should allow users to activate/deactivate the reception of real-time updates about the state (on/off) of all the lights they can control."*
- Define a high-level UML Sequence Diagram to describe how the current architecture could accommodate this requirement.
- Highlight the main disadvantage emerging from the sequence diagram.

Realizing NewReq



- The current architecture **does not** support updates in **push** mode.
- The `SmartphoneApp` carries out a continuous polling process to retrieve the status of all the lights even in case it does not change.
- This propagates also internally to `GlobalKit` and the involved `LocalKits`, thus resulting in a potentially significant communication overhead.

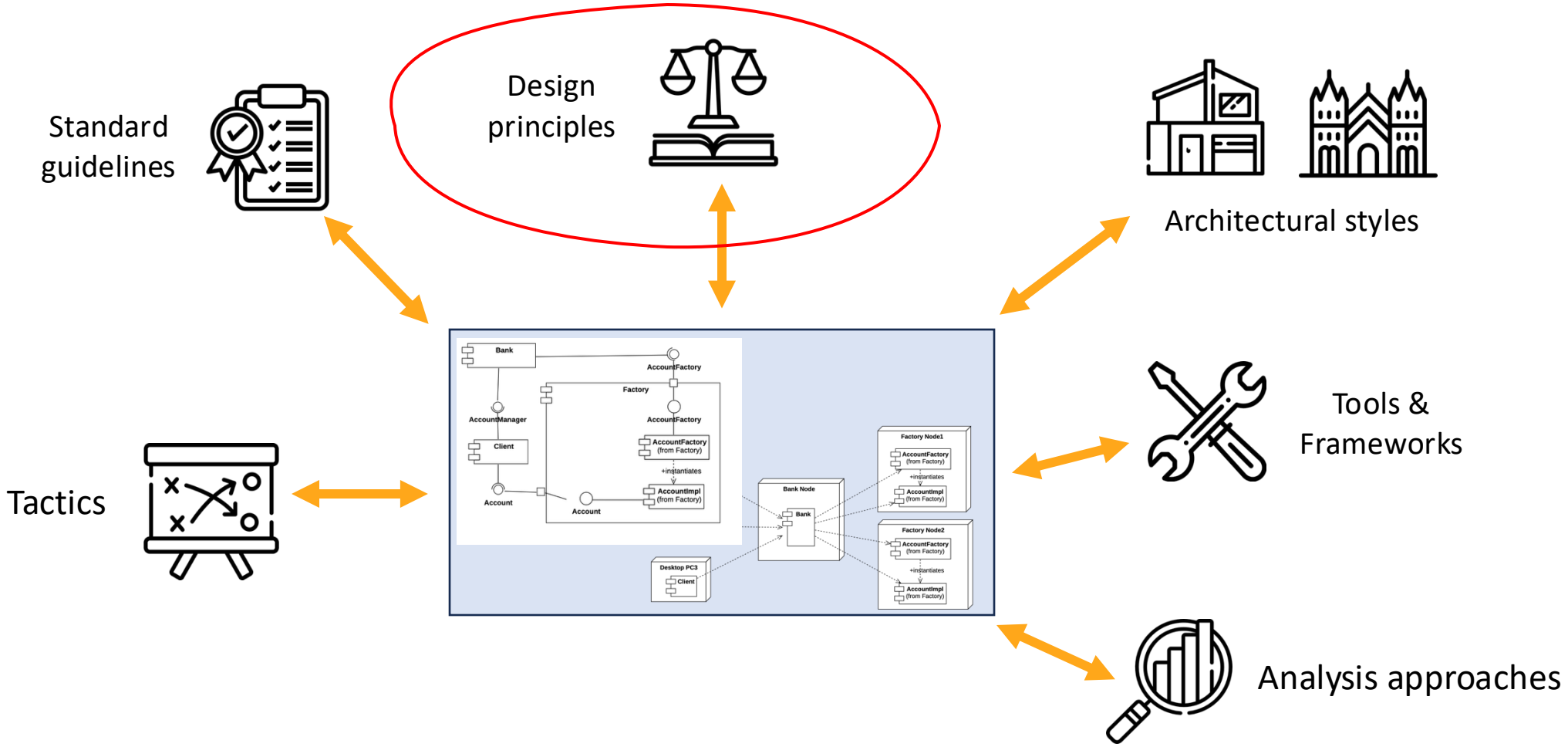
What did we learn from this exercise?

POLITECNICO MILANO 1863

- C&C structures **enable**
  - Identify and specify operations
  - Decide logical flow
  - Reason on the impact of changes
  - Identify new architectural options to address specific problems
  - Infer which components are/can be replicated and how they could be allocated to execution environments

Essential to start development!

What are the tools we have to develop and reason on software architectures?



Design principles and patterns

POLITECNICO MILANO 1863

Main principles

- Divide and Conquer
- Abstraction
- Cohesion, Encapsulation, and Coupling
- Reuse and Reusability
- Flexibility, Portability, and Obsolescence
- Testability and Defensive Design

Main patterns

- Creational, e.g., factory
- Structural, e.g., adapter, decorator
- Behavioral, e.g., state, observer
- Concurrency, e.g., thread pool

Reference textbook M. Camilli, E. Di Nitto, A. Fuggetta, A. Margara, M.G. Rossi, D.A. Tamburri. Software Engineering: a Structured Journey from Inception to Delivery. BUP 2025 [Chapters 11, 12]

References

- M. Camilli, E. Di Nitto, A. Fuggetta, A. Margara, M.G. Rossi, D.A. Tamburri. Software Engineering: a Structured Journey from Inception to Delivery. BUP 2025 [Chapters 9, 11, 12]
- Bass, Len, Paul Clements, and Rick Kazman (2021). Software Architecture in Practice. SEI Series in Software Engineering. Pearson Education.